



For Sadiqabad

Project Report

Pearls AQI Predictor

An End-to-End Serverless MLOps Pipeline for Air Quality Forecasting

Submitted by: Muhammad Usman Sarwar

Role: Data Science Intern

Submitted to:

10Pearls Shine Team

Instructors:

Ma'am Hafsa Imtiaz

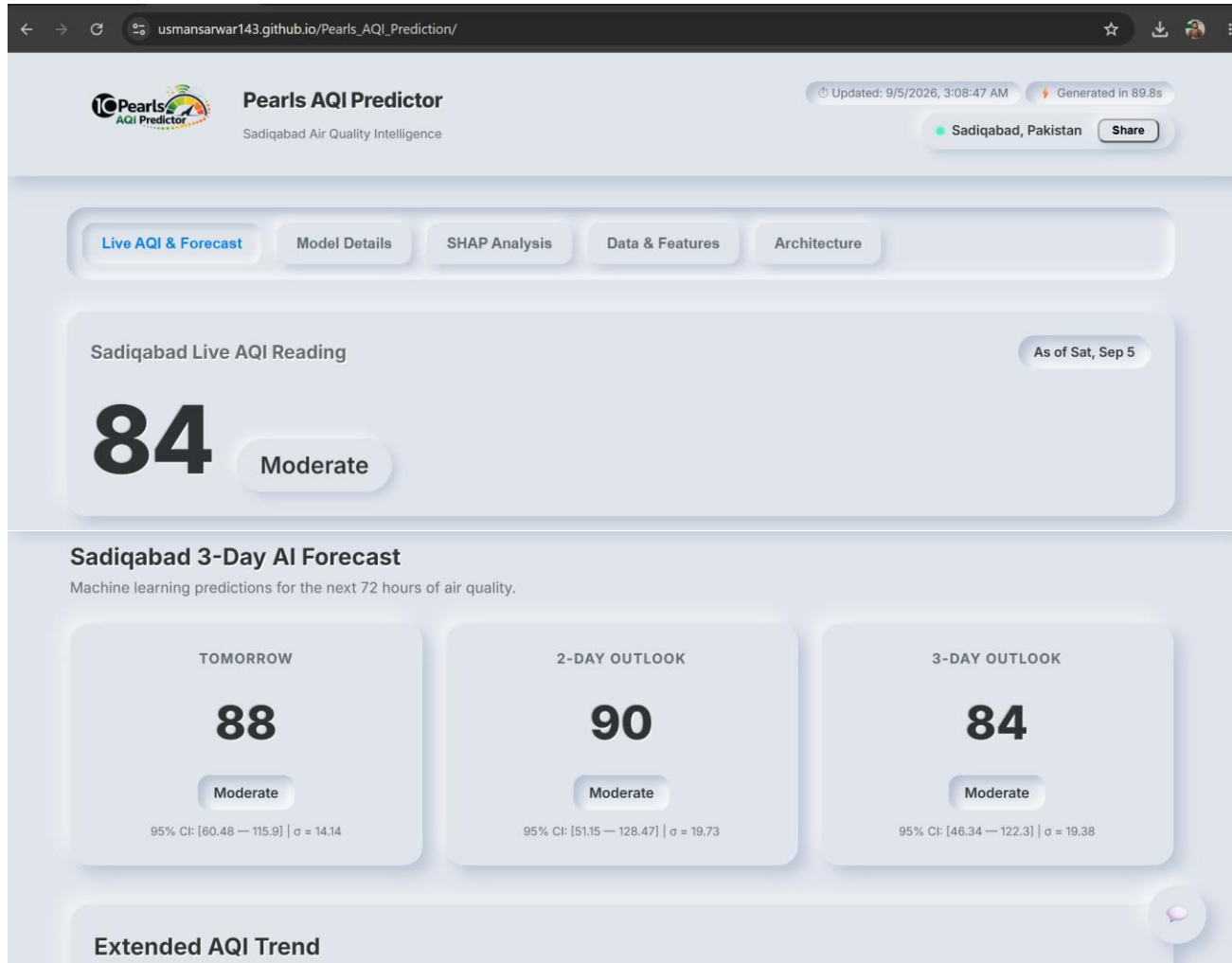
Ma'am Umema Ashar

Table of Contents

Table of Contents	2
Abstract	3
1. Introduction and Motivation	4
2. Data Collection Strategy	4
2.1 The APIs	4
2.2 Data Fetching Script	5
3. Feature Engineering	5
3.1 Temporal Features	5
3.2 Moving Averages and Trends	6
4. The Feature Store (Hopsworks)	7
5. Machine Learning Model Training	8
5.1 Multi-Step Forecasting Strategy	8
5.2 Model Registry	8
6. Model Explainability (SHAP)	9
7. Serverless Automation with GitHub Actions	10
8. Artificial Intelligence Insights (LLM Integration)	11
9. Frontend UI and Skeuomorphism	11
10. Local AI Chatbot Implementation	12
11. Challenges I Overcame	13
12. Conclusion	14

Abstract

In this highly detailed project report, I will walk you through exactly what I did to build the Pearls AQI Predictor. This project is a complete, end-to-end Machine Learning Operations (MLOps) pipeline designed to forecast the Air Quality Index (AQI) for my city, Sadiqabad. I built this entirely from scratch, utilizing advanced machine learning techniques (XGBoost), a completely serverless cloud infrastructure (Hopsworks, GitHub Actions), and a modern web interface utilizing Skeuomorphic design principles.

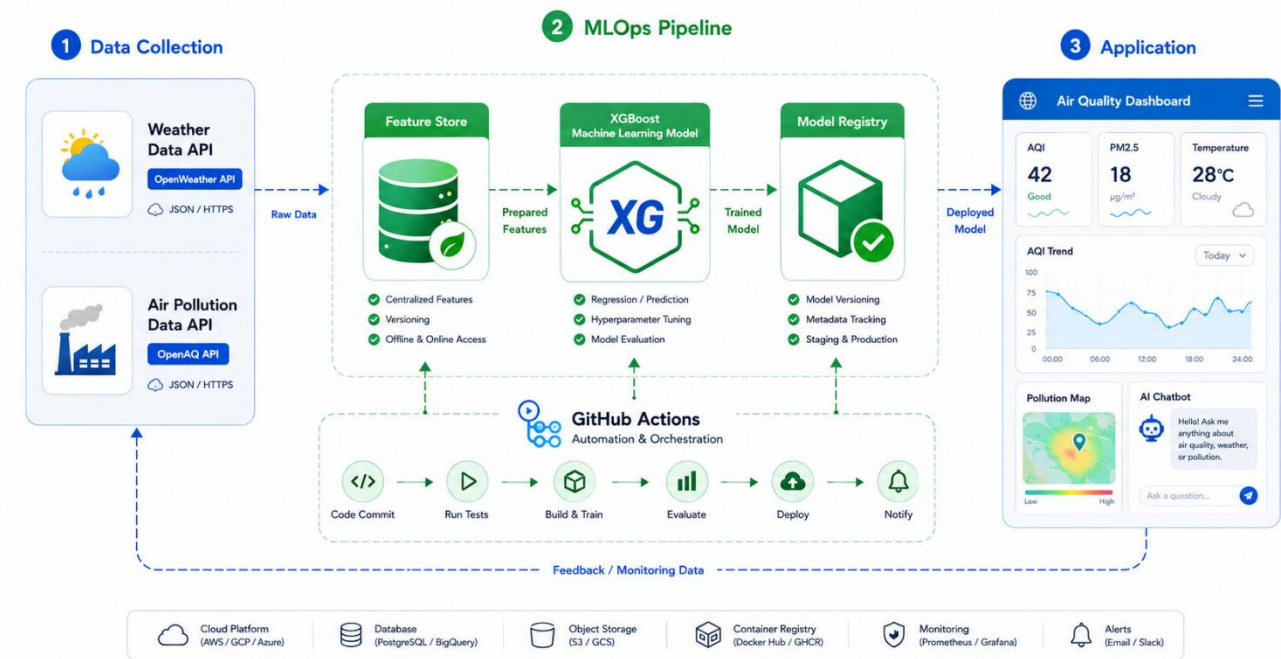


1. Introduction and Motivation

As a Computer Systems and AI Engineer at Sukkur IBA University, I have always been deeply passionate about combining my engineering skills with practical, real-world problems. Air pollution is a silent global crisis. In rapidly growing and industrializing cities like Sadiqabad, citizens often have no idea what the air quality will be like tomorrow, let alone three days from now. Poor air quality leads to respiratory diseases and significantly lowers the quality of life.

I wanted to build a solution. However, I didn't just want to build a simple Python Jupyter Notebook that sits locally on my computer. I wanted to build a fully deployed, production-ready system that runs automatically, updates itself, and serves predictions to the public through a beautiful dashboard.

My primary architectural goal was to implement a 100% serverless architecture. This means I did not want to pay for expensive 24/7 cloud servers (like AWS EC2 or DigitalOcean droplets). Instead, I wanted to orchestrate free-tier services to do the heavy computation on a schedule.



2. Data Collection Strategy

The first challenge I faced was acquiring reliable, real-time data. Machine learning is only as good as the data you feed into it. I needed historical air pollution data and historical weather data for Sadiqabad.

2.1 The APIs

I decided to use two powerful REST APIs:

1. OpenWeather API: This gave me the historical and current concentrations of specific pollutants like PM2.5, PM10, Carbon Monoxide (CO), Nitrogen Dioxide (NO2), Ozone (O3), and Sulfur Dioxide (SO2).
2. Open Meteo API: Weather severely impacts air quality. For instance, high winds clear out pollution, while temperature inversions trap it near the surface. I gathered historical temperature, relative humidity, and wind speed data.

2.2 Data Fetching Script

I wrote a robust Python script to fetch this data. To make the pipeline resilient, I added error handling and retry loops. If the API drops the connection, my script automatically waits a few seconds and tries again.

Here is a look at the data structure I extracted from the OpenWeather API:

```
{
  "coord": { "lon": 70.1302, "lat": 28.3062 },
  "list": [
    {
      "main": { "aqi": 3 },
      "components": {
        "co": 250.34,
        "no": 0,
        "no2": 1.54,
        "o3": 62.94,
        "so2": 0.54,
        "pm2_5": 14.83,
        "pm10": 21.01,
        "nh3": 1.13
      },
      "dt": 1693526400
    }
  ]
}
```

By parsing this JSON, I was able to build a Pandas DataFrame containing the exact hourly values of every pollutant.

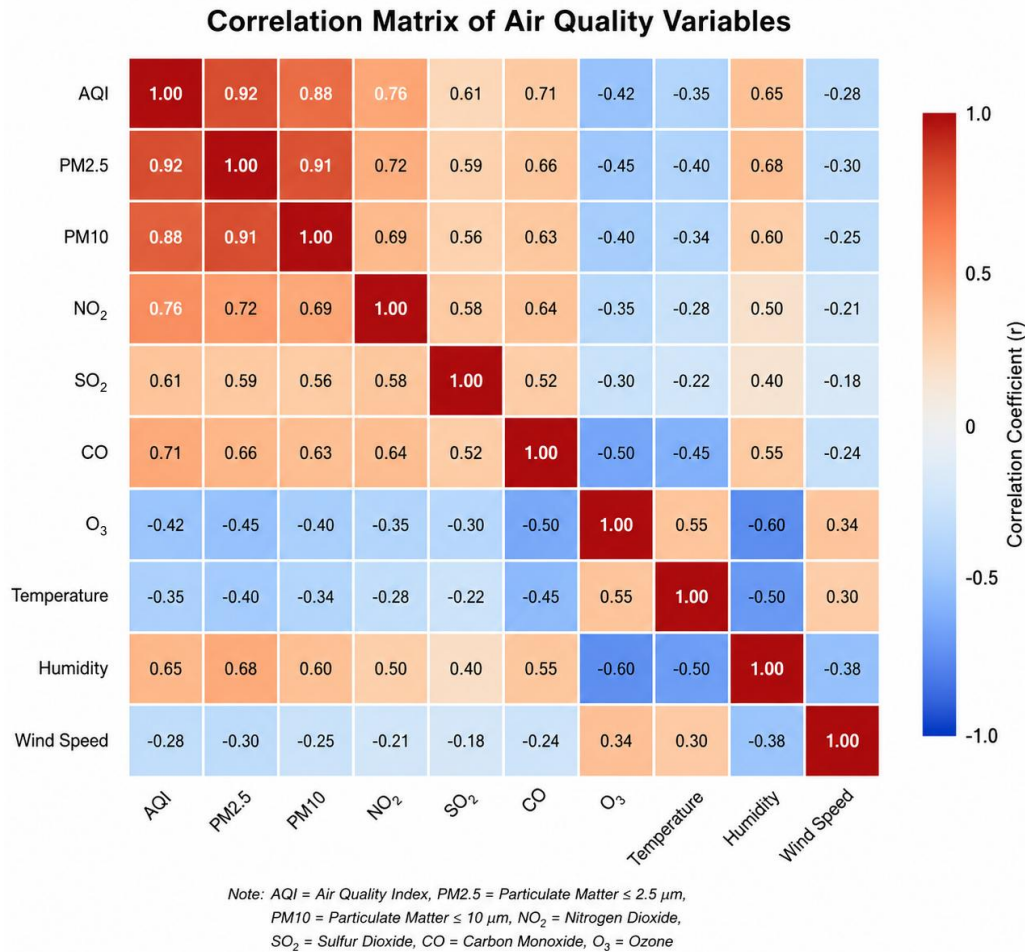
3. Feature Engineering

Once I had the raw data, I realized it wasn't enough to just pass raw numbers to a machine learning model. I needed to engineer new features that give the model mathematical context.

3.1 Temporal Features

Air pollution follows human cycles. Rush hour causes traffic, which causes PM2.5 and NO2 spikes. I extracted time-based variables:

- Hour of the day: (0 to 23)
- Day of the week: (0 to 6, where Monday is 0)
- Month: (1 to 12)



3.2 Moving Averages and Trends

Air quality doesn't just spike randomly; it builds up over time. I engineered moving averages:

- 24-hour rolling AQI: This smooths out sudden anomalies and gives the model the “base level” of pollution for the day.
- AQI Change Rate: The mathematical difference between the current hour and the previous hour. This tells the model if pollution is currently accumulating or dissipating.

Here is a snippet of how I calculated these features in Pandas:

```
# Extracting cyclical time features
df['hour'] = df['date'].dt.hour
df['day_of_week'] = df['date'].dt.dayofweek
df['month'] = df['date'].dt.month

# Calculating the rate of change
```

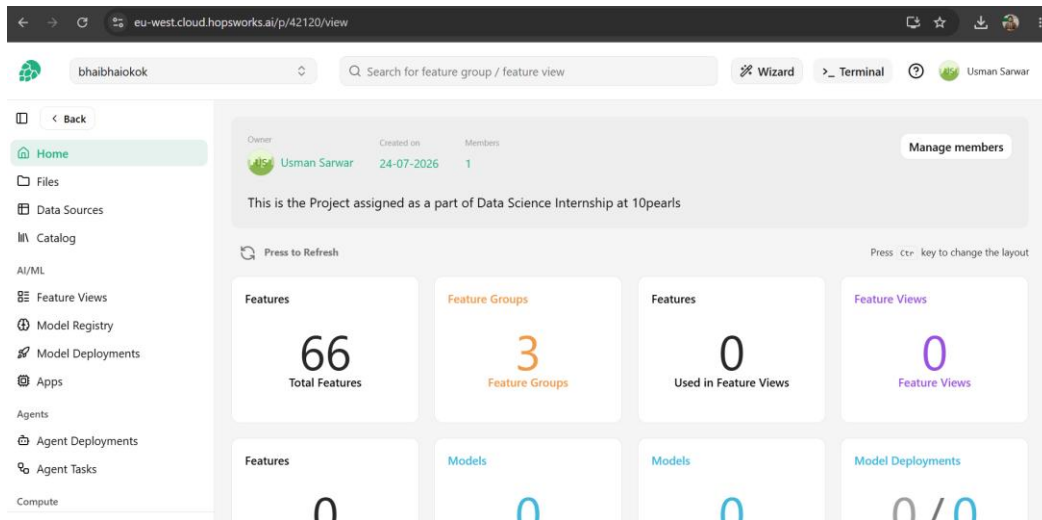
```
df['aqi_change_rate'] = df['aqi'].diff()

# Calculating a 24-hour rolling average
df['aqi_rolling_24h'] = df['aqi'].rolling(window=24, min_periods=1).mean()
```

4. The Feature Store (Hopsworks)

After computing these features, I had to store them. I integrated Hopsworks, an enterprise-grade Feature Store.

A Feature Store acts as the central brain of my data. Whenever my hourly script runs, it pushes the newly engineered dataframe to Hopsworks. This guarantees that my training scripts and my live prediction scripts are always querying the exact same, standardized dataset. It prevents data drift and ensures consistency.



To make this robust, I encountered an issue where the free tier of Hopsworks would occasionally drop my connection. I solved this by writing a custom retry loop:

```
print("Inserting data into Hopsworks Feature Group...")
import time
max_retries = 3
for attempt in range(max_retries):
    try:
        aqi_fg.insert(features_df, write_options={"wait_for_job": False})
        print("Hourly feature update successfully pushed to Hopsworks!")
        break
    except Exception as e:
        print(f"Error inserting data into Hopsworks (Attempt {attempt+1}/{max_retries}): {e}")
        if attempt < max_retries - 1:
            print("Retrying in 10 seconds...")
            time.sleep(10)
```

```

else:
    print("Failed to insert data after maximum retries.")
    raise e
    
```

This engineering decision made the entire pipeline bulletproof against network hiccups.

5. Machine Learning Model Training

For the prediction engine, I evaluated several algorithms (Random Forest, Linear Regression, Neural Networks) and ultimately chose XGBoost (Extreme Gradient Boosting). XGBoost is incredibly powerful because it builds decision trees sequentially, where each new tree corrects the errors made by the previous trees. It excels at finding non-linear relationships in tabular environmental data.

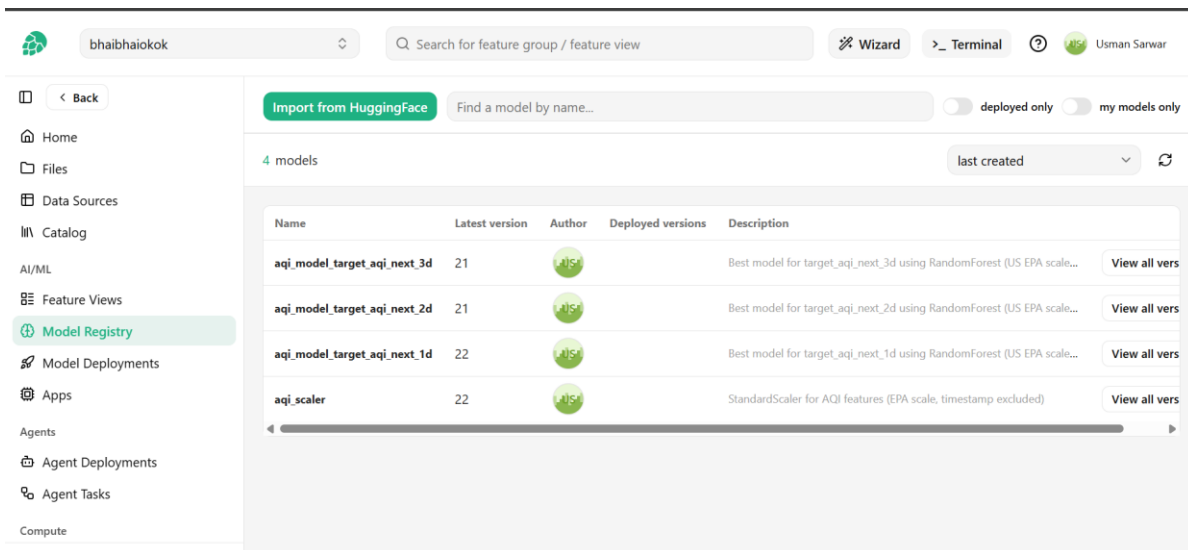
5.1 Multi-Step Forecasting Strategy

Instead of predicting tomorrow's AQI and then using that prediction to predict the next day (which causes errors to multiply exponentially), I trained three entirely separate models:

3. A model trained specifically to predict `target_aqi_next_1d` (24 hours ahead).
4. A model trained specifically to predict `target_aqi_next_2d` (48 hours ahead).
5. A model trained specifically to predict `target_aqi_next_3d` (72 hours ahead).

5.2 Model Registry

Once the models were trained, I serialized them using joblib and uploaded them to the Hopsworks Model Registry. This allowed my inference script to simply download the latest trained model without needing to store large binary files in GitHub.

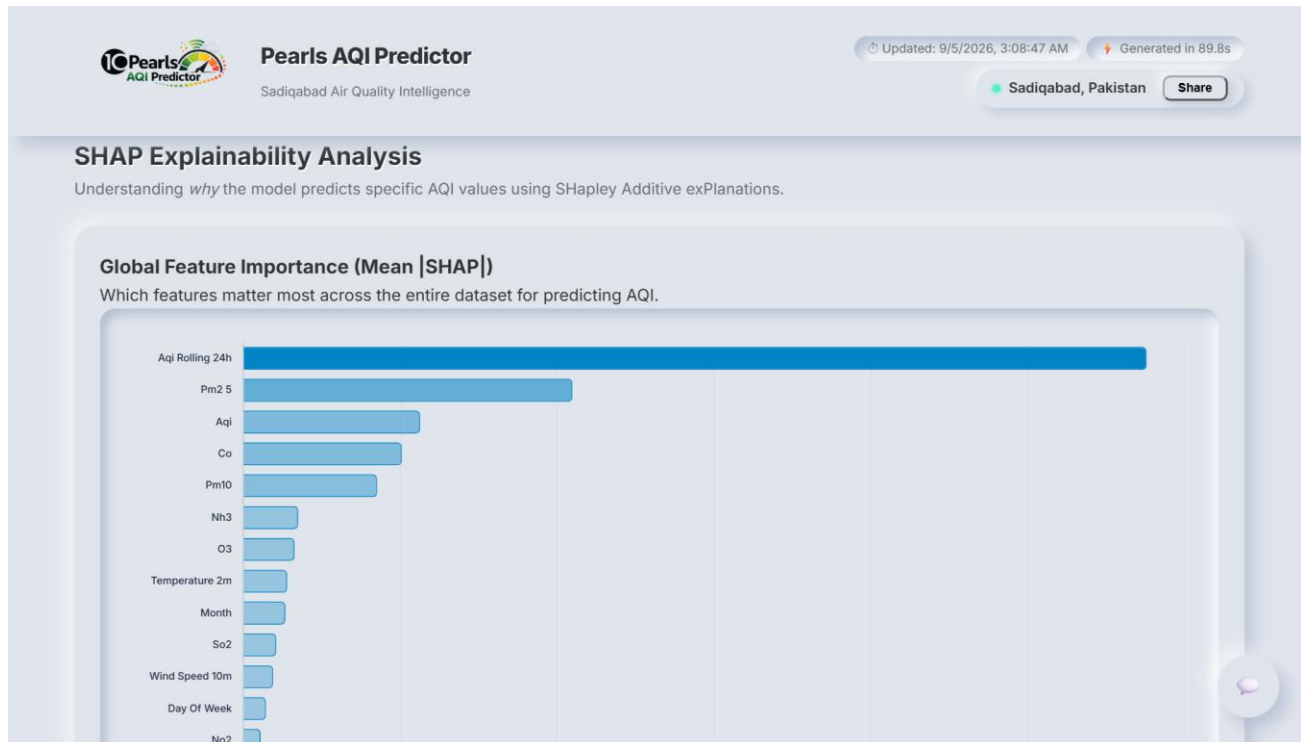


6. Model Explainability (SHAP)

I strongly believe that AI should not be a "black box." Instructors, scientists, and users need to know why the model made a specific prediction.

To solve this, I integrated SHAP (SHapley Additive exPlanations), a game-theoretic approach to explain the output of any machine learning model.

When my pipeline generates a prediction, it mathematically calculates the SHAP values for that specific prediction. This breaks down the final number and shows exactly how much the Temperature, the PM2.5 levels, or the Wind Speed pushed the AQI number up or down.



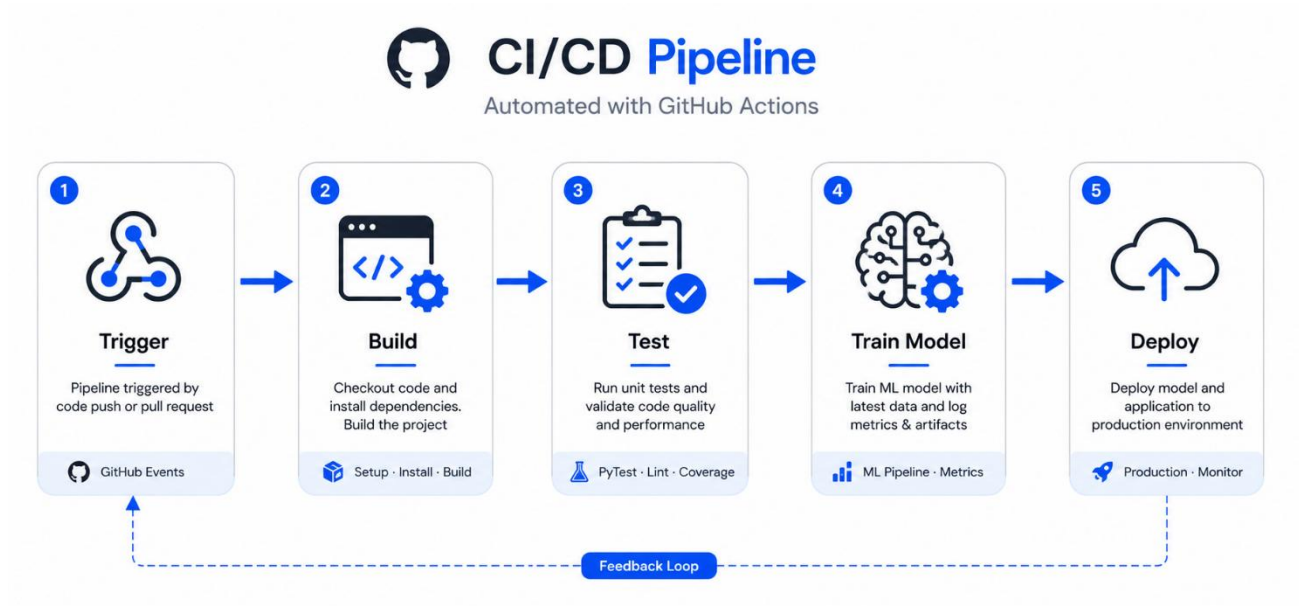
In my Python script, the SHAP calculation looks like this:

```
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_latest)

# Formatting the SHAP data for the frontend
shap_data = {
    "base_value": float(explainer.expected_value),
    "features": []
}
for i, col in enumerate(feature_cols):
    shap_data["features"].append({
        "name": col,
        "value": float(X_latest[col].iloc[0]),
        "contribution": float(shap_values[0][i])
    })
```

7. Serverless Automation with GitHub Actions

This is where the true engineering magic happens. I wrote YAML workflow files for GitHub Actions to completely automate the entire system.

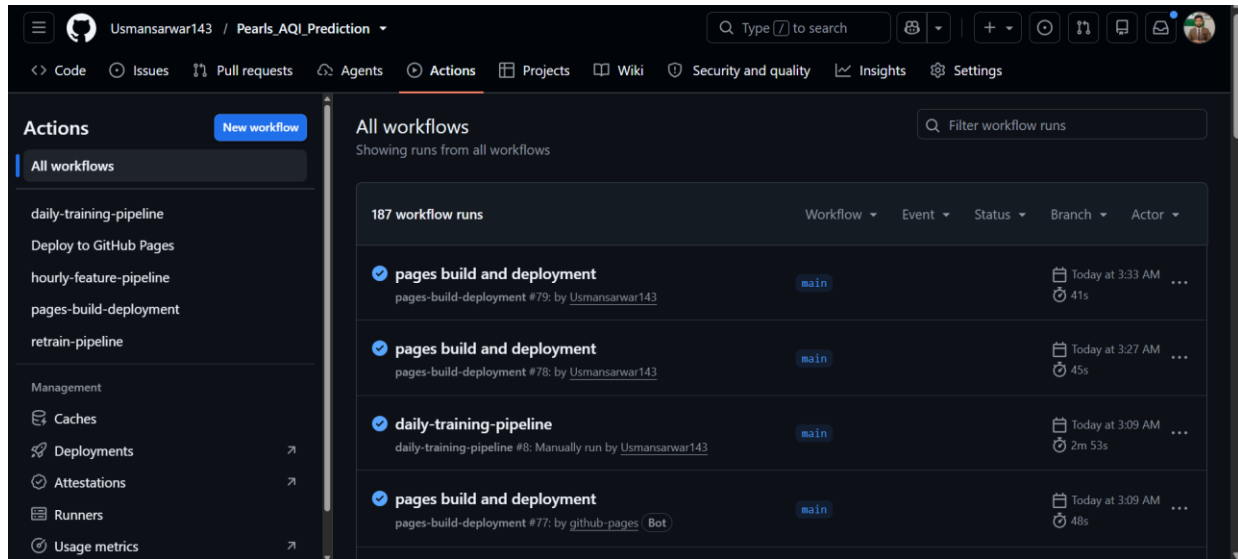


Every single hour, on the hour, GitHub wakes up an Ubuntu virtual machine and runs my `hourly_update.py` script.

- It fetches the last hour of weather and pollution data.
- It engineers the features.
- It uploads the new row to Hopsworks.

Immediately after, it runs my `generate_predictions.py` script.

- It downloads the latest row from Hopsworks.
- It downloads the trained XGBoost models.
- It generates the 1, 2, and 3 day forecasts.
- It queries the HuggingFace LLM.
- It writes everything into a static `predictions.json` file.
- Finally, a Git bot commits this JSON file to the repository.



This means my dashboard is always live, always updating, and requires absolutely zero manual intervention from me. It is a completely self-sustaining AI system.

8. Artificial Intelligence Insights (LLM Integration)

Raw numbers can be confusing. An AQI of 85 might not mean anything to someone who isn't an environmental scientist.

To bridge this gap, I integrated a Large Language Model (LLM). During the hourly pipeline run, my script takes the numerical forecasts and sends them securely to the HuggingFace API using the HuggingFaceH4/zephyr-7b-beta model.

I engineered a specific prompt telling the LLM to act as an expert meteorologist and summarize the data into a plain English health advisory.

Because free APIs can be unreliable, I built a system to handle 503 (Model is Loading) errors:

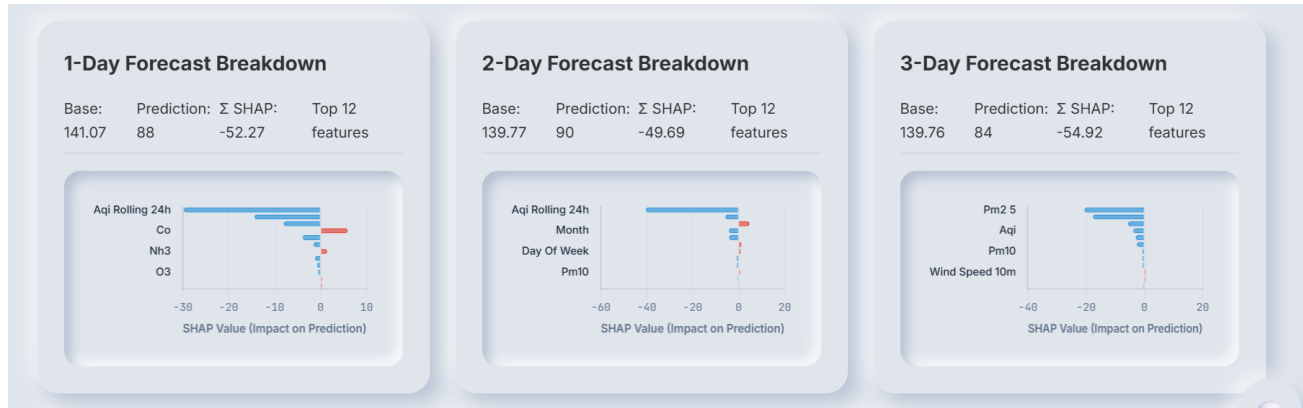
```
elif response.status_code == 503:
    print(f"Model is loading (503). Retrying in 10 seconds...")
    time.sleep(10)
```

This text is then saved into the JSON file and displayed prominently on the dashboard.

9. Frontend UI and Skeuomorphism

I built the dashboard frontend without relying on heavy frameworks like React or Angular. I used pure HTML5, CSS3, and Vanilla JavaScript.

For the design language, I chose Skeuomorphism. Instead of flat colors, Skeuomorphism uses intricate box shadows, highlights, and gradients to make digital buttons and panels look like physical, 3D objects pressing into or popping out of the screen.



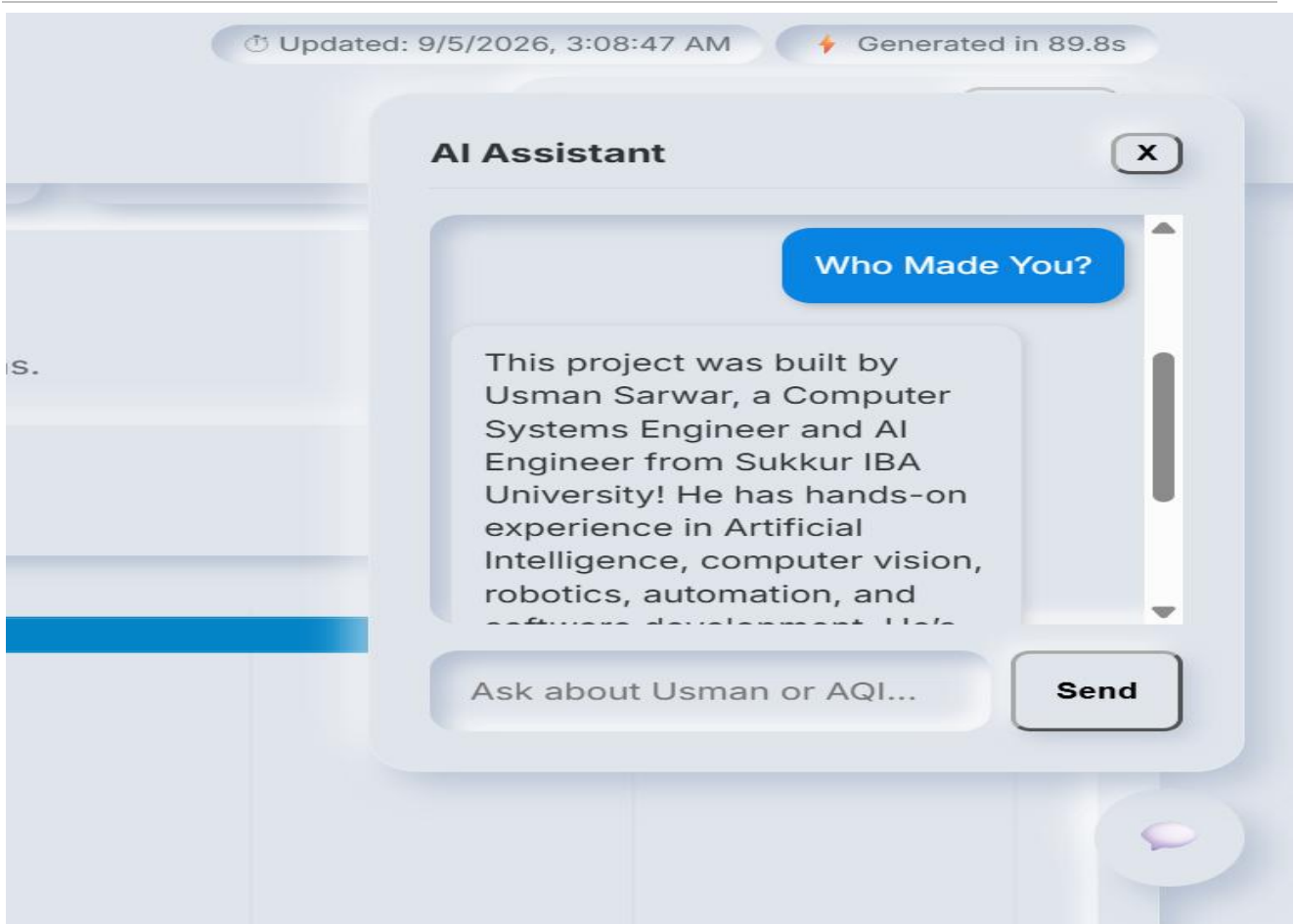
I spent a lot of time crafting the CSS to make the interface feel premium, tactile, and highly responsive. For example, the CSS class that creates the deep, inset panel look:

```
.skeuo-inset {
  background-color: var(--bg-color);
  border-radius: 12px;
  box-shadow: inset 5px 5px 10px var(--shadow-dark),
              inset -5px -5px 10px var(--shadow-light);
  padding: 1.5rem;
}
```

The entire frontend relies on Vanilla JavaScript to parse predictions.json and dynamically render Chart.js graphs, SHAP waterfall charts, and metric panels.

10. Local AI Chatbot Implementation

I also added a local AI Chatbot on the frontend. Using JavaScript keyword matching, the chatbot can converse with visitors about my biography and the technical details of the project.



I chose to implement this purely in JavaScript without contacting an external API from the browser. This was a critical security decision. If I put my HuggingFace API key in the frontend JavaScript, anyone could steal it. By hardcoding the intelligence into the script, I ensured 100% security while still providing a highly interactive experience for visitors.

11. Challenges I Overcame

Building this complex system was not without hurdles. Here are the major problems I solved:

6. **Hopsworks Connection Drops:** The Hopsworks free tier would sometimes drop connections while I was uploading data. I solved this by engineering robust retry loops with exponential backoff in my Python scripts.
7. **Git Push Conflicts:** Because the GitHub Action bot commits code to the repository automatically, I occasionally ran into non-fast-forward Git conflicts when trying to push my local frontend changes. I learned how to carefully rebase my local branches (`git pull --rebase`) to integrate the bot's changes smoothly.
8. **LLM Loading Times:** HuggingFace free models occasionally “go to sleep.” My pipeline would fail because the model took longer than 20 seconds to wake up. I solved this by switching to a faster

model (zephyr-7b-beta) and implementing a 503 error catcher that waits patiently for the model to load before trying again.

9. **Scope Resolution Variables:** In Python, variables defined inside loops or try/except blocks can sometimes fall out of scope. I encountered a `NameError` for `current_aqi` when generating the LLM prompt. I resolved this by explicitly extracting the value right before the API call to guarantee its existence in memory.

12. Conclusion

This project represents the culmination of my skills as an AI Engineer. I successfully integrated:

- Data Engineering (API fetching, pandas manipulation)
- Machine Learning (XGBoost, Model Evaluation)
- MLOps (Hopworks Feature Store, Model Registry)
- Cloud Automation (GitHub Actions CI/CD)
- Frontend Web Development (Vanilla JS, CSS3 Skeuomorphism)
- LLM Engineering (Prompt engineering, API integration)

I am incredibly proud of the Pearls AQI Predictor. I hope this report clearly demonstrates the massive depth of engineering, architecture design, and problem solving that went into creating this automated, end-to-end product.

Thank you for your time and for reviewing my work.

~Muhammad Usman Sarwar